CS 166 ASSIGNMENT 2 SOLUTION

Overview

- Marks on question:
  - No mark: 1-part short answer question (the key point(s) for explanation is underlined)
  - !: purely auto-graded question (answer boxed or color-coded)
  - +: muli-part question with auto-graded parts and explanation parts.
    The answers for auto-graded parts are boxed, the key point(s) for explanation is underlined.
- Grading rules:
  - Auto-graded questions are all-or-nothing
  - 0.5 pt short answers are all-or-nothing
  - Other short answers questions: -0.5 for each key point missing.
  - If your answer is totally wrong, you will get 0.5 points (on whole question, not each sub-question).

Week 3 – Public Key Cryptos – Parts 1 & 2 (Q1 – Q8)

!Q1: Usages of public key cryptos (L5 P5 & P6)

I. For encryption, should use receiver's public key to encrypt and receiver uses private key to decrypt.

- Alice encrypts message with Bob's public key
- Bob decrypts it with Bob's private key.

II. For signature, signee uses private key to encrypt and others use signee's public key to decrypt to verify.

- Alice encrypts message with Alice's private key
- Bob decrypts it with Alice's public key.

!Q2: Public key crypto for digital signature (L5 P6)

To verify a signature, Bob needs to get the signed message = $[M]_{Alice}$, and use Alice's public key to decrypt to verify that $\{[M]_{Alice}\}_{Alice} = M$. That is, Bob also needs to know the original message M to verify.

+Q3: RSA example with N = 35 (L5 P17 – 18)

Q3.1 & 3.2: e must be relatively prime to $\phi(N) = (p - 1) * (q - 1) = (5 - 1) * (7 - 1) = 24$,
so from the options, 5, 7, and 25 all can be possible e for N = 35.

Q3.3: The smallest e in Q3.1 is 5.

I. $d = e^{-1} \bmod \phi(N)$, since $5 * 5 \bmod 24 = 1$, d can be 5. (can be other d's as long as $d = e^{-1} \bmod \phi(N)$).

II. To encrypt, should use the public key e, that is, $\{M\}_e = M^e \bmod N$.

III. To sign, should use the private key d, that is, $[M]_d = M^d \bmod N$.

Yan Chen
Computer Science Department

Q4: Cube root attack on RSA when e = 3 (L5 P19 – 20)

I. (Mention one of two possibilities is enough)

Possibility 1: when M^e = $M^3 < N$, then $C = M^3 \bmod N = M^3$. That is, attacker can easily get $M = \sqrt[3]{c}$.

Possibility 2: send same message M to 3 users using e = 3, so $C_1 = M^3 \bmod N_1$, $C2 = M^3 \bmod N_2$, and $C_3 = M^3 \bmod N_3$. By Chinese remainder theorem, can get $C = M^3 \bmod N_1N_2N_3$. Since $N = N_1N_2N_3$ should be a very big number, M^3 is very likely to be < N. The rest is the same as possibility 1.

II. Given (N, e) = (33, 3) and d = 7.
If M = 3, since $M^3 = 3^3 = 27 < N = 33$, $C = M^3$, so cube root attack is possible;
If M = 4, since $M^3 = 4^3 = 64 > N = 33$, $C \neq M^3$, so cube root attack is not possible.

+Q5: Double encryption using same N for RSA (L5 P12, P19 – 20)

Q5.1: $\boxed{\text{False}}$, this double encryption will not increase the security.

Q5.2: Since $C_0 = M^{e_0} \bmod N$ and $C_1 = C_0^{e_1} \bmod N$, we get $C_1 = (M^{e_0} \bmod N)^{e_1} \bmod N$

Then based on ((a mod n) (b mod n)) mod n = ab mod n, we can get $((a \bmod n)^x) \bmod n = a^x \bmod n$,

That is, $C_1 = M^{e_0 e_1} \bmod N$. This is same as using another $e = e_0e_1$. This only increase the size of e.
But the size of e will not affect the security of the RSA. The security of RSA is highly related to the size of N.

+Q6: Diffie-Hellman (L6 P7 – 8)

Based on the Diffie-Hellman algorithm, Bob computes $N^b \bmod p$ where N is from Alice and b is Bob's private key (can omit mod p for this question).

+Q7: MIM attack on Diffie-Hellman (L7 P5 & P6)

Q7.1: $\boxed{\text{False}}$, MIM will not be successful.

Q7.2: Although Trudy can get $g^{at} \bmod p = (g^a \bmod p)^t \bmod p$ and $g^{bt} \bmod p = (g^b \bmod p)^t \bmod p$,
she cannot compute $g^{abt} \bmod p$ using $g^{at} \bmod p$ and $g^{bt} \bmod p$ without knowing a or b.

$(g^{abt} \bmod p = (g^{at})^b \bmod p = (g^{bt})^a \bmod p \neq (g^{at} \bmod p)(g^{bt} \bmod p))$

Q8: ECC pros & cons (L6 P11)

I. Advantage: smaller key size is needed to achieve the same level of security. Or more secure with the same key size.

II. Since the math is too complex, not everyone can fully understand it to analyze the system (e.g., pick a secure key) or no formal proof that it works.

Week 4 – Public Key Cryptos Part 1 & Hash Functions (Q9 – Q15)

!Q9: Common misinterpretation of public key cryptos (L7 P5 – 6)

False. Signature CANNOT identify the sender. The signed message $[M]_{Alice}$ is public to everyone, and everyone can encrypt M by Bob's public key, so, everyone can send $\{[M]_{Alice}\}_{Bob}$, not just Alice herself.

Q10: Data integrity and non-repudiation (L7 P4)

I. Non-repudiation means signee cannot deny the signature later.

II. No, MAC doesn't provide non-repudiation since it is a symmetric key crypto – the key is shared between the two parties that are communicating.

III. Yes, digital signature using public key crypto provides non-repudiation, since the signee is the ONLY one who knows his/her private key.

Q11: Digital certificates (L7 P7)

I. A digital certificate contains the username, and his/her public key.

II. Can also include other info such as birthday, blood type, etc. (or any info that's reasonable)

III. Minimize the amount, so the certificate won't be too big. Also, no need to expose that much unnecessary personal information (or any reason that's reasonable).

!Q12: Collisions in hash functions (L8 P4 & P9)

False. Collision definitely will happen since the input space is larger than the output space.
For a secure hash function, collisions still happen, but it is hard to find any collisions by computation.

!Q13: Hash function to reduce spam emails (L8 P19 – 20)

False. The usage mention in class will only make it harder/more costly to send spam emails, NOT to detect and block them.

Q14: Hash used in online-bid system (L8 P20)

I. Two properties involved

- One-way, that is, given h(bid), hard to find the bid.
- (Weak) Collison Resistance, That is, knowing the bid and h(bid), bidders cannot easily find another bid' ≠ bid such that h(bid') = h(bid). (Or bidders cannot deny or change their bid afterwards).

II. The hash is secure since Trudy can only do a brute-force attack to try all the hashes to find collisions.
But if Trudy can guess the range of the bids, then she can just hash those bids in the range.
This means she has fewer hashes to compute, which is possible to finish in a reasonable amount of time.

Q15: Analyze collision for a n-bit hash (L8 P5 – 9)

n-bit hash means there are 2^n possible hashes.

I. To find 1 collision:
Suppose x hashes are needed, then the number of comparisons (x choose 2) = ${}_xC_2 = x^2$.
Solving $x^2 = 1 * 2^n$, we get x = $\sqrt{2^n} = 2^{n/2}$ hashes to find 1 collision.

II. To find m collisions:
Suppose x hashes are needed, then the number of comparisons (x choose 2) = ${}_xC_2 = x^2$.
Solving $x^2 = m * 2^n$, we get x = $\sqrt{m * 2^n} = \sqrt{m} * 2^{n/2}$ hashes to find m collisions.

Note that this means it gets progressively easier to find collisions as more hashes are computed.
This makes sense since it is not a fixed-ratio mapping such as every 2 inputs map to the same hash.
Instead, maybe 3 inputs map to hash1, 4 inputs map to hash2, 10 inputs map to hash3, etc.
That is, each new hash is compared to all of the previous hashes.

III. If compute x hashes:
Suppose y collisions will be found, then ${}_xC_2 = x^2 = y * 2^n$.
Solving the equation, we get y = $x^2 / 2^n$ collisions will be found if compute x hashes.

(Or explaining this is the inverse of II. That is, II computes x given m, while III computes m given x)

Yan Chen
Computer Science Department